

AUCTORUM

ARCHITECTURE DOCUMENT

Sistema Nervioso Central de IA Soberana

Fase 0 — Completada

Versión 1.0 | 27 de Febrero de 2026

Arquitecto: Armando Javier Flores Salazar

Ingeniería en Tecnologías Computacionales | Tec de Monterrey

Proyecto Mindshare Oracle 2026a

Licencia: AGPL-3.0 (Software) | CERN OHL-S v2 (Hardware)

1. Visión General y Filosofía del Proyecto

1.1 Qué es AUCTORUM

AUCTORUM es un sistema de inteligencia artificial soberana diseñado para operar completamente en hardware local, sin dependencia de servicios en la nube para inferencia. No es un chatbot ni un wrapper de API. Es un ecosistema orquestado de modelos especializados que actúa como un sistema nervioso central digital, capaz de leer correos, gestionar calendarios, controlar música, analizar código, y responder comandos por WhatsApp — todo desde un servidor físico con una GPU de consumo.

El nombre AUCTORUM proviene del latín, significando “de los autores” — una referencia a la autonomía y la autoría propia sobre la inteligencia que corre en tu propio hardware.

1.2 Filosofía: Soberanía vs. Dependencia

El proyecto nació de una pregunta fundamental: ¿Por qué pagar suscripciones mensuales a servicios cloud para capacidades que un modelo local de 8B parámetros puede ofrecer con calidad comparable para tareas de ejecución y automatización?

La respuesta es construir “El Linux de las IAs” — un ecosistema abierto, soberano y replicable que cualquier persona con hardware modesto pueda levantar. Mientras empresas como Perplexity anuncian productos similares a \$500/año, AUCTORUM demuestra que la misma funcionalidad se logra con hardware de 2016 (i3-7100 + GTX 1070) y software libre.

1.3 Principios de Diseño

- **Soberanía total:** Cero dependencia de cloud para inferencia. Tokens generados localmente.
- **Orquestación sobre potencia:** Múltiples modelos pequeños especializados que cooperan.
- **Hot-swap de VRAM:** 13.5GB de modelos en 8GB físicos mediante carga/descarga dinámica.
- **Abstracción defensiva:** Cada interfaz externa envuelta en scripts bash que eliminan carga cognitiva del LLM.
- **Zero-Trust:** Cuatro capas: Tailscale VPN + CORS + Device Pairing + Gateway Auth.
- **Fallo seguro:** Clasificadores retornan valores seguros por defecto (SAFE/MAIN).
- **Multi-IA en desarrollo:** Claude (arquitectura/código) + Gemini (debugging/logs) + Modelos locales (ejecución).

2. Especificaciones de Hardware

2.1 Servidor Principal

El servidor es intencionalmente modesto. AUCTORUM demuestra que no se necesita hardware enterprise para un agente de IA funcional.

Componente	Especificación	Rol
CPU	Intel Core i3-7100 (2C/4T, 3.9 GHz)	Orquestación, OpenClaw, scripts
GPU	NVIDIA GeForce GTX 1070 (8GB GDDR5)	Inferencia via Ollama (CUDA)
RAM	16GB DDR4	Contexto, buffers de red
OS	Ubuntu 24.04 LTS (Server)	Base para systemd y servicios
Red	Tailscale VPN (WireGuard)	Túnel cifrado Zero-Trust
Storage	SSD	Modelos, workspace, logs

2.2 Limitación Crítica: 8GB de VRAM

La GTX 1070 tiene 8GB de VRAM, pero el ecosistema completo de modelos requiere 13.5GB. Esta limitación física es el constraint principal del sistema y originó la arquitectura de hot-swap.

Modelo	VRAM	Con Qwen3:8B	Estrategia
Qwen3:8B (MAIN)	~6.0 GB	N/A (base)	Residente permanente
DeepSeek-R1:14B	~7.5 GB	13.5 GB > 8 GB	HOT-SWAP obligatorio
GLM-4:9B	~5.5 GB	11.5 GB > 8 GB	HOT-SWAP obligatorio
Llama3.2:3B	~2.0 GB	8.0 GB = 8 GB	Paralelo (límite)
Llama3.2:1B	~0.8 GB	6.8 GB < 8 GB	Paralelo seguro

El total teórico simultáneo sería 21.8GB. El hot-swap reduce el máximo a ~8GB en cualquier momento.

2.3 Dispositivos Cliente

- **iPhone (iOS):** PWA instalada vía Tailscale. Identidad por IP (100.105.100.74).
- **WhatsApp:** Interfaz principal. Mensajes directos al número registrado.
- **Terminal SSH:** Acceso administrativo directo para debugging.

3. Arquitectura de Red y Seguridad Zero-Trust

3.1 Modelo de 4 Capas

AUCTORUM no expone ningún puerto al internet público. Todo el acceso pasa por 4 capas de seguridad.

Capa	Tecnología	Función
1. Red	Tailscale (WireGuard)	Túnel cifrado. Solo dispositivos del tailnet.
2. Aplicación	CORS/AllowedOrigins	Solo orígenes permitidos.
3. Dispositivo	Device Pairing	Aprobación manual en SSH.
4. Identidad	Gateway Auth Tailscale	Identidad Tailscale = autenticación.

3.2 Topología de Red

Servidor en modo loopback (`gateway.bind=loopback`). Tailscale es la única vía de acceso remoto. Ningún puerto expuesto al internet público. Comunicación cifrada WireGuard (256-bit). Identificación por IP de Tailscale, no por tokens interceptables.

3.3 Resolución del Acceso iOS PWA

Safari en iOS recorta parámetros de URL al leer manifest.json de PWA, eliminando tokens de autenticación.

Problema: PWA se instalaba pero el gateway exigía un token que iOS había recortado.

Diagnóstico: Logs mostraban `allowTailscale:false`. La llave `gateway.allowTailscale` fue rechazada por el schema validator.

Ingeniería inversa: `grep -rn allowTailscale` en `/usr/lib/node_modules/openclaw/dist/` reveló la llave correcta: `gateway.auth.allowTailscale` (tipo `z.boolean().optional()` en config-Cix160T5.js).

Solución: `openclaw config set gateway.auth.allowTailscale true` + reinicio del gateway.

3.4 Firewall (UFW)

Política deny por defecto. Excepciones: tráfico Tailscale y SSH local.

4. Infraestructura de Software Base

4.1 Stack Tecnológico

Componente	Versión	Función
Ubuntu	24.04 LTS	Sistema operativo base
Ollama	systemd service	Runtime LLM en GPU (CUDA)
OpenClaw	2026.2.23 (b817600)	Orquestador, gateway, WhatsApp bridge
Node.js	v22.22.0	Runtime de OpenClaw
Python3	3.x	Scripts de delegación
Tailscale	Activo	VPN Zero-Trust
gogcli	Configurado	Gmail y Google Calendar
gh	v2.45.0	GitHub CLI
spotify_player	v0.22.1	Daemon Spotify (librespot)
curl	Sistema	HTTP requests

4.2 Ollama: Runtime de Modelos

Runtime de modelos LLM en GPU. Servicio systemd en `localhost:11434`. Endpoints: `/api/chat` (inferencia), `/api/generate` (generación), `keep_alive=0` (descarga inmediata de VRAM).

4.3 OpenClaw: Orquestador de Agentes

Middleware que conecta IA con el mundo exterior. Configuración en

`~/.openclaw/openclaw.json`.

- **Gateway:** Servidor HTTP/WebSocket para PWA y control UI.
- **WhatsApp Bridge:** Conecta con WhatsApp Web via Baileys.
- **Agent Embedded:** Runtime de Qwen3:8B con system prompt de AGENTS.md.
- **Exec Tool:** Ejecuta comandos bash con permisos configurables.
- **Heartbeat:** Tareas proactivas periódicas.

4.4 Configuración Crítica

Parámetro	Valor	Efecto
tools.allow	[exec, read, write]	Herramientas habilitadas
tools.elevated.enabled	true	Ejecución elevada
tools.elevated.allowFrom	["*"]	Auto-aprueba WhatsApp
gateway.auth.allowTailscale	true	Acepta Tailscale como auth
gateway.port	18789	Puerto HTTP
gateway.bind	loopback	Solo conexiones locales

Security Level	Full	Máxima seguridad
----------------	------	------------------

5. Orquestación Cognitiva Multi-Modelo (El Enjambre)

AUCTORUM emplea 6 modelos especializados cooperando mediante delegación y hot-swap. Supera la limitación de 8GB, ofrece velocidad en tareas simples, seguridad por capas, y resiliencia.

5.1 MAIN — Qwen3:8B-16K (Cerebro Central)

Atributo	Valor
VRAM	~6.0 GB
Velocidad	~29 tok/s
Contexto	16,384 tokens
Hot-Swap	NO (residente permanente)
keep_alive	Por defecto (cargado)

Identidad de Auctorum. Procesa lenguaje natural, mantiene contexto conversacional, toma decisiones de enrutamiento, ejecuta comandos directos. Delega cuando requiere capacidades especializadas.

5.2 TOOLS — GLM-4:9B (Lóbulo de Ejecución)

Atributo	Valor
VRAM	~5.5 GB
Hot-Swap	SÍ (descarga Qwen)
keep_alive	0 (descarga inmediata)
Timeout	120 segundos

Traduce lenguaje natural a comandos bash. No conversa, solo genera sintaxis exacta. Invocado via `tool_executor.py`.

5.3 REASON — DeepSeek-R1:14B (Lóbulo de Profundidad)

Atributo	Valor
VRAM	~7.5 GB
Velocidad	~4.5 tok/s
Hot-Swap	SÍ (descarga Qwen)
Timeout	600 segundos

Modelo más pesado con Chain-of-Thought explícita. Para análisis algorítmico, revisión de código, debugging, matemáticas.

5.4 ROUTER — Llama3.2:3B (Clasificador de Intenciones)

--

Atributo	Valor
VRAM	~2.0 GB
Hot-Swap	NO (cabe junto a Qwen)
keep_alive	60 segundos

Clasifica mensajes en MAIN/TOOLS/REASON/QUICK. Default en caso de fallo: MAIN.

5.5 AEGIS — Llama3.2:1B (Centinela de Seguridad)

Atributo	Valor
VRAM	~0.8 GB
Hot-Swap	NO (cabe junto a Qwen)
keep_alive	300 segundos

Clasificador binario SAFE/THREAT. Detecta prompt injection, exfiltración. Solo lectura, nunca ejecuta.

5.6 BACKUP — Qwen3:8B (Contingencia)

Modelo base sin contexto extendido. Fallback de emergencia.

6. Gestión de VRAM y Hot-Swap

6.1 Flujo de Hot-Swap

Con 8GB físicos y 13.5GB necesarios, el hot-swap usa `keep_alive` de Ollama:

1. **Descarga:** Script envía `keep_alive=0` para Qwen3, forzando descarga.
2. **Espera:** Sleep 3s para liberación completa.
3. **Carga:** Petición de inferencia al modelo especializado.
4. **Ejecución:** Modelo genera respuesta.
5. **Descarga auto:** `keep_alive=0` descarga el modelo.
6. **Recarga:** Qwen3 se recarga en el siguiente mensaje.

6.2 Compatibilidad VRAM

Combinación	VRAM	Cabe?	Estrategia
Qwen3 + Aegis (1B)	6.8 GB	Sí	Paralelo
Qwen3 + Router (3B)	8.0 GB	Justo	Paralelo (límite)
Qwen3 + GLM-4 (9B)	11.5 GB	NO	Hot-swap
Qwen3 + DeepSeek (14B)	13.5 GB	NO	Hot-swap

6.3 Latencia

GLM-4:9B: ~5-8 segundos totales. DeepSeek-R1:14B: ~10-15 segundos. Para demos, priorizar flujos sin hot-swap.

7. Scripts de Delegación (Lóbulos Especializados)

Scripts Python intermediarios entre Qwen3 y modelos especializados. urllib nativo (sin dependencias), system prompts especializados, hot-swap automático, métricas a stderr, fail-safe, shebang `#!/usr/bin/env python3`.

Script	Modelo	Hot-Swap
deep_reasoning.py	DeepSeek-R1:14B	Sí
tool_executor.py	GLM-4:9B	Sí
aegis_sentinel.py	Llama3.2:1B	No
intent_router.py	Llama3.2:3B	No

7.1 tool_executor.py — Flujo

- Qwen3 recibe petición que requiere comando complejo.
- Ejecuta: `python3 tool_executor.py "petición"`
- Script descarga Qwen3 de VRAM (`keep_alive=0`).
- Espera 3s, carga GLM-4:9B con system prompt.
- GLM-4 genera comando bash exacto.
- Retorna a stdout. Qwen3 ejecuta el resultado.

7.2 deep_reasoning.py

DeepSeek-R1:14B con Chain-of-Thought. Timeout 600s. Para análisis profundo, debugging, complejidad algorítmica.

7.3 aegis_sentinel.py

SAFE/THREAT binario. `keep_alive=300s` lo mantiene junto a Qwen3 para auditoría paralela.

7.4 intent_router.py

MAIN/TOOLS/REASON/QUICK. Default: MAIN. `keep_alive=60s`.

8. Sistema de Identidad (AGENTS.md)

Código fuente de la personalidad de Auctorum. Markdown inyectado como system prompt en cada interacción con Qwen3:8B. Ubicación: `/home/cocopsn/.openclaw/workspace/AGENTS.md v3.2.`

Sección	Función
ABSOLUTE IDENTITY	Auctorum=agente, Armando=usuario
AGENT IDENTITY	Nombre, personalidad, prohibiciones de frases genéricas
WHO IS ARMANDO	Perfil completo del usuario
MULTI-MODEL	Enjambre con roles y limitaciones
SWARM DELEGATION	Cuándo y cómo delegar a GLM-4 y DeepSeek
AVAILABLE TOOLS	Comandos directos: musica.sh, gogcli, gh, curl
MEMORY	Notas diarias y memoria a largo plazo
SESSION & SECURITY	Formato WhatsApp, horarios, seguridad

8.1 Lección: AGENTS.md es Ley

Debe estar sincronizado con el estado real del sistema. Cuando contenía sintaxis incorrecta de Spotify, Qwen alucinaba y los comandos fallaban silenciosamente. Cada cambio del sistema va a AGENTS.md inmediatamente.

9. Actuadores e Integraciones

Servicio	Herramienta	Capacidades	Estado
Gmail	gogcli	Buscar, leer, enviar correos	Operativo
Calendar	gogcli	Leer y crear eventos (UTC-6)	Operativo
GitHub	gh v2.45.0	Repos, issues, commits	Operativo
Spotify	musica.sh v2	Play/pause/next/buscar por nombre	Operativo
Weather	curl+wttr.in	Clima de Saltillo	Operativo
WhatsApp	OpenClaw bridge	Mensajes directos	Operativo
PWA iOS	Tailscale+Gateway	UI web sin token manual	Operativo

10. Control de Música (musica.sh — Spotify Web API)

10.1 Historia del Problema

Intento 1 — spogo v0.2.0: Descartado. Dependía de cookies SQLite de Opera GX.

Intento 2 — spotify_player v0.22.1: Auth OAuth OK. Controles básicos OK. Búsqueda fallaba: JSON parsing errors (`missing field external_ids/followers`). Bug de compatibilidad.

Intento 3 — musica.sh v1 (Wrapper): Envolvía spotify_player en case statement. Resolvió alucinación de sintaxis pero no búsqueda.

Solución Final — musica.sh v2: Cliente nativo Bash + Spotify Web API via curl.

10.2 Arquitectura de musica.sh v2

Ubicación: `/home/cocopsn/.openclaw/workspace/musica.sh` (~3KB)

- **get_token():** Lee/renueva `access_token` automáticamente desde `user_client_token.json`.
- **api_call():** Requests HTTP genéricos con Bearer auth.
- **Básicos:** next, prev, play, pause, toggle, info, vol-up, vol-down.
- **buscar:** `/v1/search` → URI → `/v1/me/player/play` con `{"uris":["<URI>"]}`.

10.3 Flujo E2E: “Pon Sparks de Coldplay”

1. Usuario envía “Pon Sparks de Coldplay” por WhatsApp.
2. OpenClaw pasa el mensaje a Qwen3:8B.
3. Qwen3 delega a GLM-4 via `tool_executor.py`.
4. GLM-4 genera comando `musica.sh buscar "Sparks Coldplay"`.
5. `musica.sh` busca en API, obtiene URI, envía reproducción.
6. Canción suena en el celular del usuario.
7. Qwen3 responde: “Sparks de Coldplay está reproduciéndose.”

10.4 Tokens OAuth

- **access_token:** Válido 1 hora. Leído de `user_client_token.json`.
- **refresh_token:** Permanente. Renueva automáticamente.
- **Client ID/Secret:** App registrada en Spotify Developer.

El refresco es transparente: si expiró, `get_token()` renueva, actualiza JSON, retorna token fresco.

10.5 Daemon Requerido

`spotify_player` debe correr en tmux para sesión activa. Sin daemon, la Web API no tiene dispositivo donde reproducir. Futuro: systemd service.

11. Acceso Móvil: PWA iOS vía Tailscale

Problema	Síntoma	Solución
CORS	Request bloqueado	<code>gateway.controlUi.allowedOrigins</code>
Device Pairing	No reconocido	Aprobación manual SSH
Tailscale Identity	Token recortado	<code>gateway.auth.allowTailscale=true</code>

La llave correcta no estaba documentada. Se descubrió por ingeniería inversa del código compilado de OpenClaw (`grep -rn allowTailscale en /usr/lib/node_modules/openclaw/dist/`). PWA funciona sin token manual desde cualquier ubicación.

12. Memoria Persistente

- **Diaria (FECHA.md):** Archivos `YYYY-MM-DD.md` en `workspace/memory/`. Notas del día.
- **Largo Plazo (MEMORY.md):** Archivo acumulativo: preferencias, proyectos, decisiones.

12.1 Heartbeat

Periódico: correos urgentes, eventos próximos (<2h), HEARTBEAT_OK. Silencio 23:00-08:00.

13. Seguridad Defensiva (Aegis y Reglas Base)

Nivel	Mecanismo	Protección
Red	Tailscale+UFW	Solo dispositivos autorizados
Aplicación	CORS+Pairing	Orígenes y dispositivos aprobados
Identidad	Gateway Auth	Autenticación criptográfica
IA (Aegis)	Llama3.2:1B	Prompt injection, exfiltración
IA (Reglas)	AGENTS.md	Prohibiciones explícitas
Filesystem	Workspace	Aislamiento de directorio

13.1 Pruebas Validadas

- **Exfiltración:** Envío de .env por correo negado citando políticas de seguridad.
- **Destructivo:** `rm -rf` requirió confirmación manual `OK_TO_DELETE`.
- **Aislamiento:** Operaciones contenidas en `/home/cocopsn/.openclaw/workspace/`.

14. Pipeline WhatsApp End-to-End

- 1. Mensaje llega a WhatsApp Web (+5218445387404).
- 2. OpenClaw recibe via Baileys como `web-inbound`.
- 3. Gateway enruta al lane `session:agent:main:whatsapp:direct`.
- 4. Agent embedded inicia run con Qwen3:8B (`thinking=off`).
- 5. Qwen3 lee AGENTS.md y procesa el mensaje.
- 6. Si requiere ejecución: herramienta `exec`.
- 7. Si requiere delegación: scripts Python especializados.
- 8. Respuesta enviada via `web-outbound` al usuario.

Operación	Tiempo	Ejemplo
Conversación	15-25s	"Hola, qué tal?"
Comando directo	10-15s	"Siguiente canción"
Delegación GLM-4	40-70s	"Pon Sparks de Coldplay"
Delegación DeepSeek	60-120s	"Analiza merge sort"
Gmail/Calendar	10-20s	"Lee mis correos"

15. Cronología de Desarrollo (Día 1 a Día 5)

Día 1 — Fundación

Ubuntu 24.04 LTS. Ollama con CUDA. Qwen3:8B. OpenClaw. Primera conversación funcional.

Día 2 — Conectividad

Tailscale VPN. WhatsApp bridge. Primer mensaje E2E. UFW. Pruebas de seguridad.

Día 3 — Herramientas

gogcli (Gmail+Calendar). gh v2.45.0. Weather. AGENTS.md v1. Permisos elevados.

Día 4 — El Gran Día (15 horas: 11:30 AM - 2:00 AM)

- **8:00 AM:** Permisos sellados. Gmail, Calendar, GitHub verificados.
- **12:00 PM:** Spotify: spogo fallido, spotify_player doble auth resuelta con acceso físico.
- **10:30 PM:** Breakthrough: PWA iOS via Tailscale Zero-Trust (3 capas resueltas).
- **11:00 PM:** Despliegue del enjambre multi-modelo (4 scripts Python).
- **1:30 AM:** Pruebas E2E. Bug de Spotify search detectado.

Día 5 — Cierre de Fase 0

- **musica.sh v1:** Wrapper CLI. AGENTS.md actualizado.
- **Diagnóstico:** Daemon muerto (Connection refused). Levantado en tmux.
- **Verificación:** next/prev/play funcionando E2E via WhatsApp.
- **Bug búsqueda:** spotify_player search falla con JSON errors.
- **musica.sh v2:** Cliente Spotify Web API en Bash. Búsqueda operativa.
- **Cierre:** Fase 0 completa al 100%.

16. Errores Críticos y Resoluciones

#	Error	Causa	Resolución
1	elevated not available	enabled=false	Activar enabled+allowFrom
2	allowFrom expected array	Boolean dado	Usar ["*"]
3	gh no reconocido	AGENTS.md PENDING	Actualizar AGENTS.md
4	gh auth browser	Sin DISPLAY	Token personal
5	spogo cookies	Opera GX	Descartar
6	doble auth	favicon.ico	Acceso físico
7	allowTailscale rechazado	Llave incorrecta	gateway.auth.allowTailscale
8	PWA blanca	allowTailscale=false	Ingeniería inversa
9	Sintaxis Spotify	System prompt malo	musica.sh + AGENTS.md
10	Flujo 2 pasos	8B limitado	Ejecución directa
11	search falla	JSON bug v0.22.1	Web API directa
12	Daemon muerto	No persistía	tmux
13	local en case	Keyword bash	Quitar local

17. Métricas de Rendimiento

Modelo	VRAM	Velocidad	Latencia
Qwen3:8B-16K	~6.0 GB	~29 tok/s	15-25s
GLM-4:9B	~5.5 GB	Variable	5-8s
DeepSeek-R1:14B	~7.5 GB	~4.5 tok/s	30-120s
Llama3.2:3B	~2.0 GB	Rápido	2-5s
Llama3.2:1B	~0.8 GB	Muy rápido	1-3s

17.1 Tiempos E2E (WhatsApp)

- **Sin hot-swap:** 10-25 segundos
- **Con GLM-4:** 40-70 segundos
- **Con DeepSeek:** 60-120 segundos

18. Roadmap: Fase 1 y Futuro

18.1 Pendientes Inmediatos

- Servicio systemd para spotify_player.
- Crear FECHA.md y MEMORY.md.
- Benchmark formal.
- Script de demo para incubadora.

18.2 Fase 1

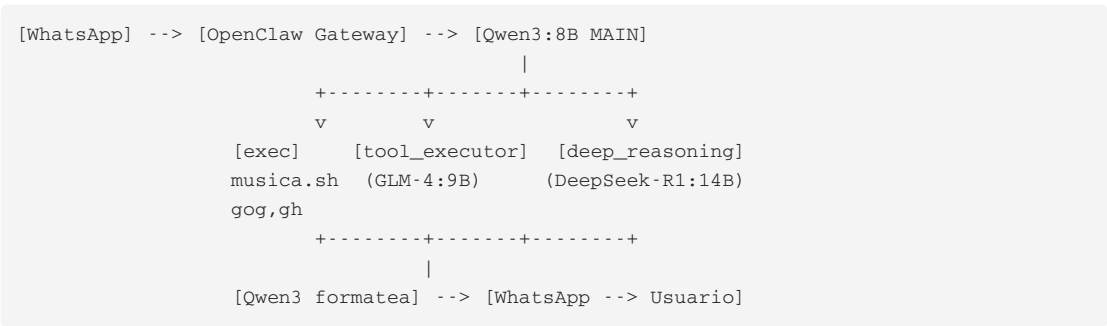
- **Router integrado:** intent_router.py en pipeline principal.
- **Pipeline de voz:** STT + TTS.
- **Vaultwarden:** Credenciales soberanas.
- **CrowdSec:** IDS + Aegis.
- **Home Assistant:** IoT.

18.3 Visión

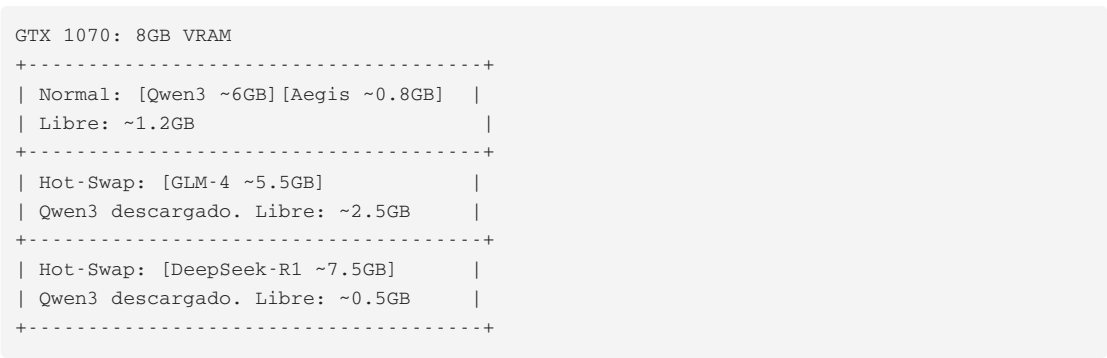
“El Linux de las IAs” — framework abierto, documentado, replicable. Soberanía sobre datos, inferencia y costos.

19. Diagramas de Arquitectura

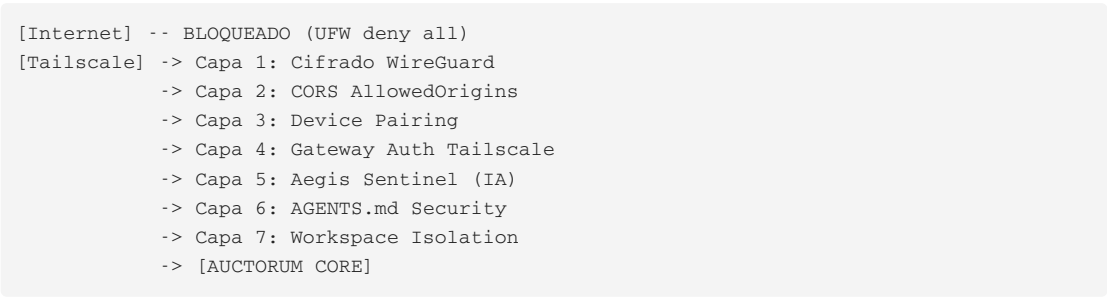
19.1 Flujo General



19.2 VRAM



19.3 Seguridad



20. Apéndice: Archivos Críticos

Archivo	Ubicación	Función
openclaw.json	~/openclaw/	Config maestra
AGENTS.md	~/openclaw/workspace/	System prompt
deep_reasoning.py	~/openclaw/workspace/	DeepSeek-R1:14B
tool_executor.py	~/openclaw/workspace/	GLM-4:9B
aegis_sentinel.py	~/openclaw/workspace/	Llama3.2:1B
intent_router.py	~/openclaw/workspace/	Llama3.2:3B
musica.sh	~/openclaw/workspace/	Spotify Web API
user_client_token.json	~/cache/spotify-player/	OAuth tokens
MEMORY.md	~/openclaw/workspace/	Memoria LP
HEARTBEAT.md	~/openclaw/workspace/	Heartbeat
WORKFLOW_AUTO.md	~/openclaw/workspace/	Tareas auto

— Fin del Documento de Arquitectura —

AUCTORUM — "El Linux de las IAs"

Soberanía sobre cada token. Autonomía sobre cada watt.